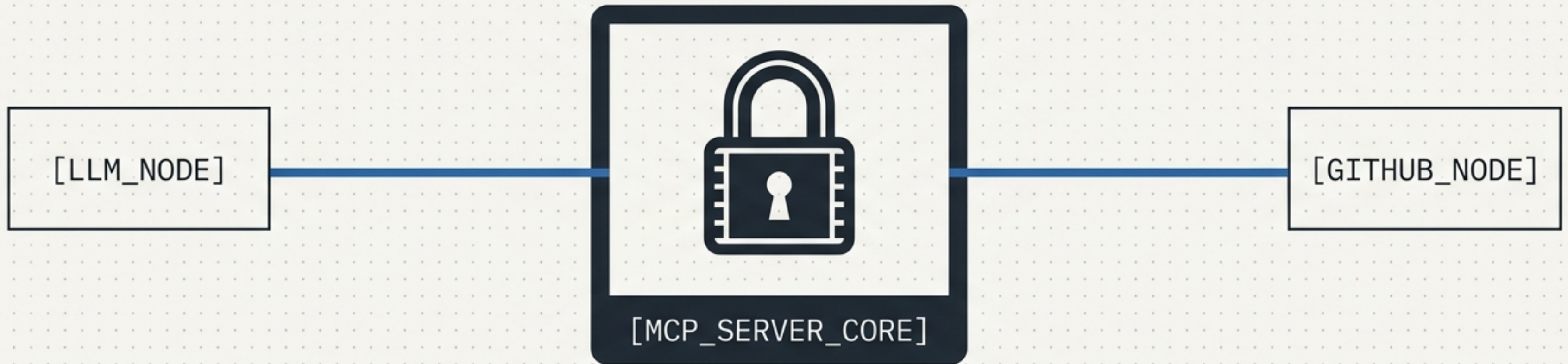


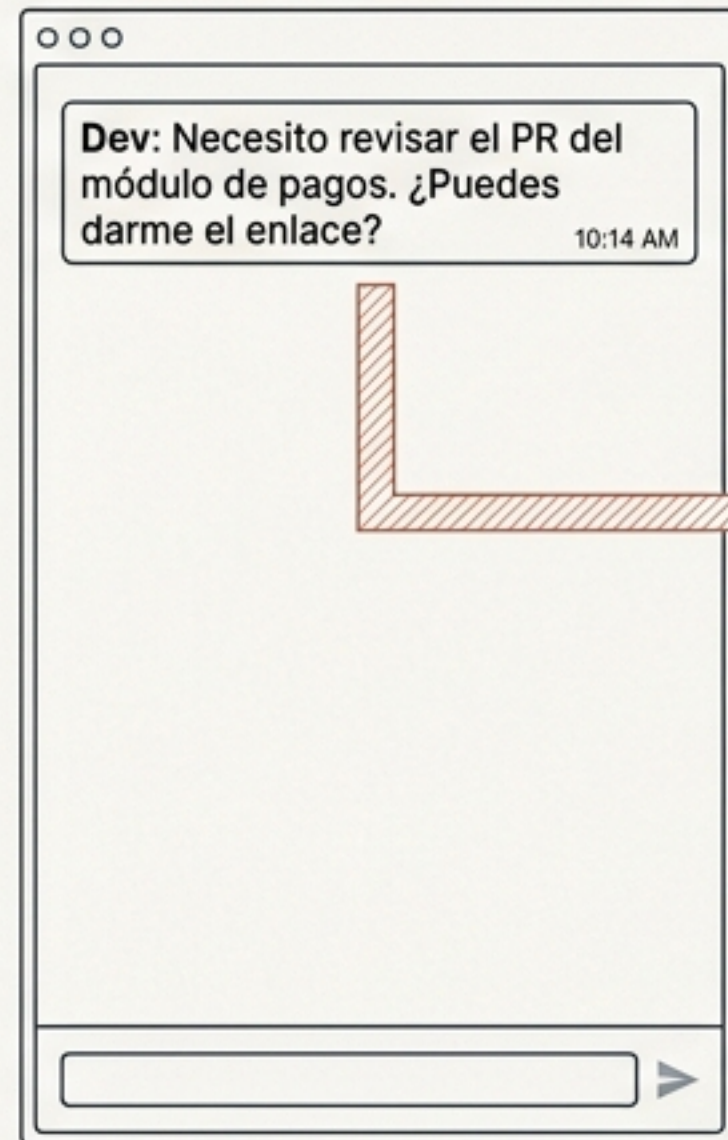
Automatización de GitHub con MCP: Arquitectura y Ejecución



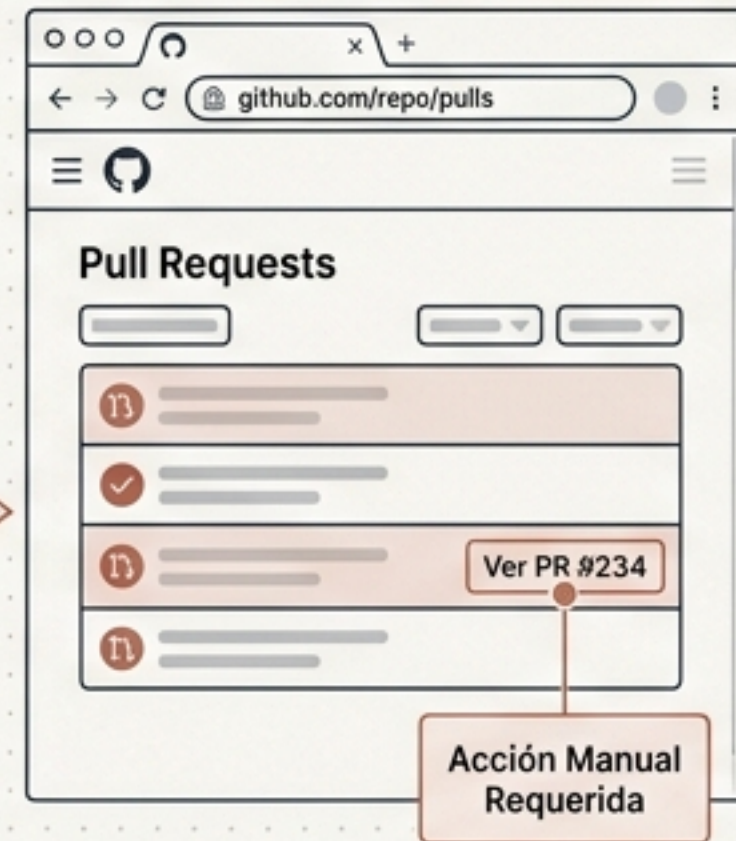
**El LLM decide qué operación necesita.
El servidor conserva el control absoluto de
la validación y ejecución.**

La automatización devuelve el foco al flujo de conversación

Chat Window (Contexto Primario)

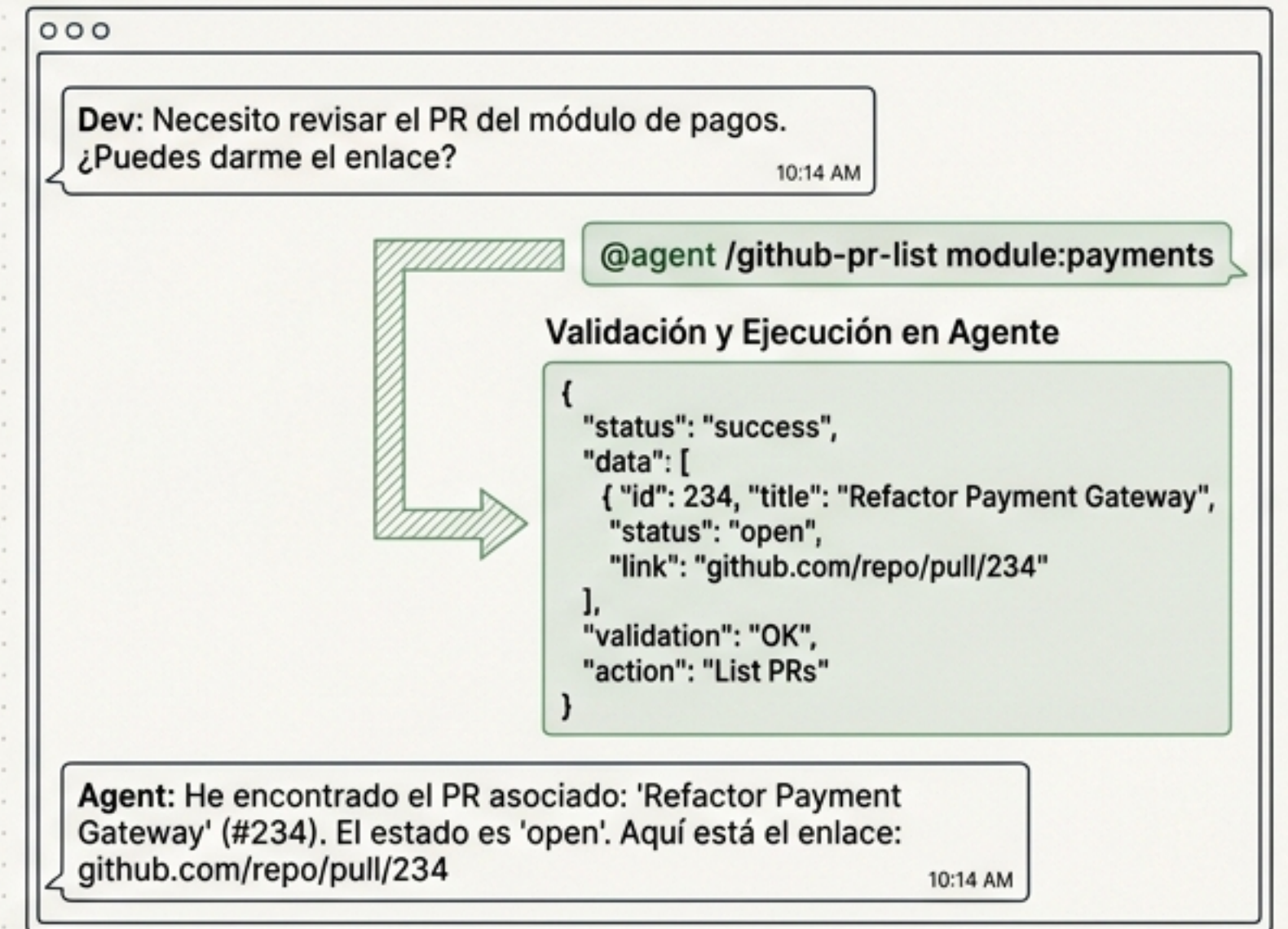


GitHub Browser (Cambio de Contexto)



El Problema: Las operaciones repetitivas en repositorios rompen el contexto cognitivo al obligar a cambiar de herramienta.

Unified Interface (Automatización Integrada)



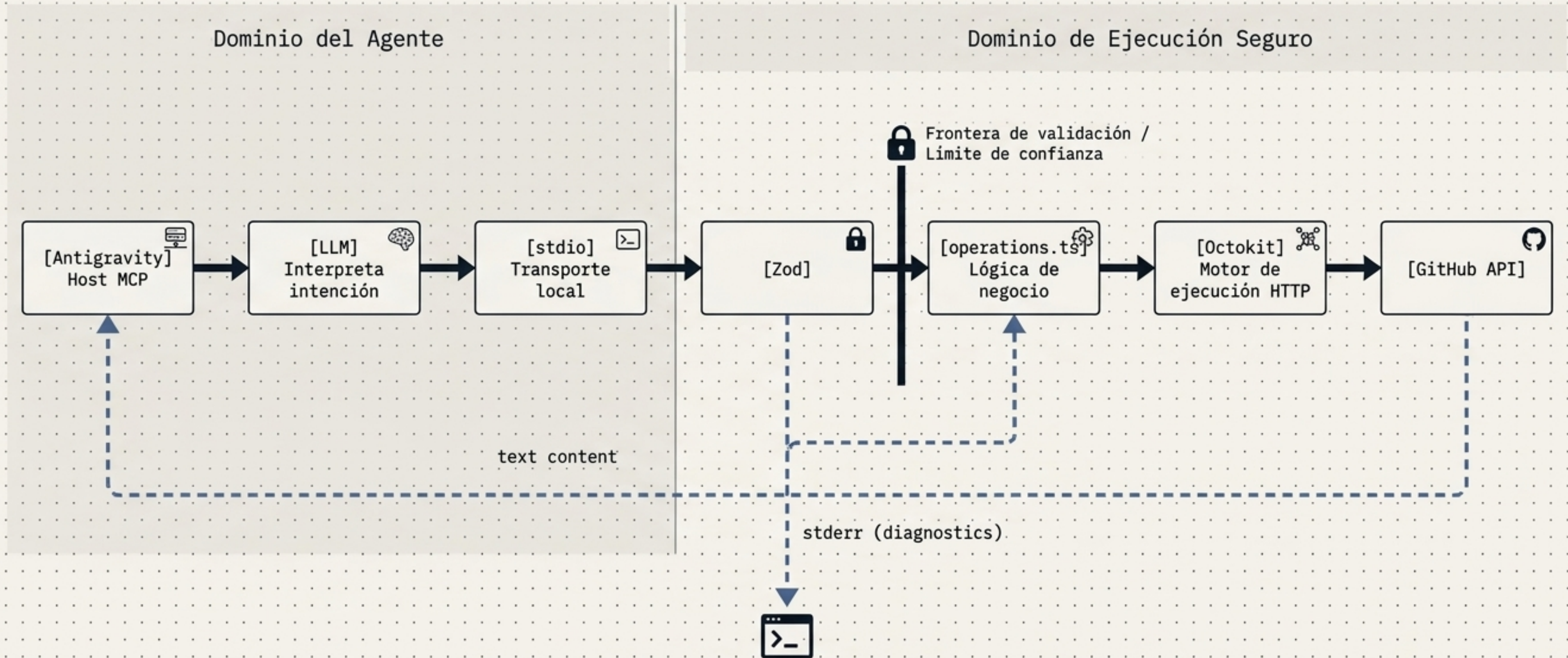
El Objetivo: Exponer operaciones frecuentes de GitHub a agentes compatibles, usando lenguaje natural, pero garantizando validación estricta y errores accionables.

El arsenal operativo: 5 herramientas con contratos estrictos

Lectura	Escritura
list_repositories Consulta repositorios del usuario autenticado.	create_repository Crea un nuevo repositorio para el usuario autenticado.
list_issues Lista issues por estado en un repositorio específico.	create_issue Abre un issue definiendo título, descripción y etiquetas.
	create_commit Crea o actualiza un archivo directamente en la rama.

Interfaz de lenguaje natural → Schema estructurado → Operación de API explícita

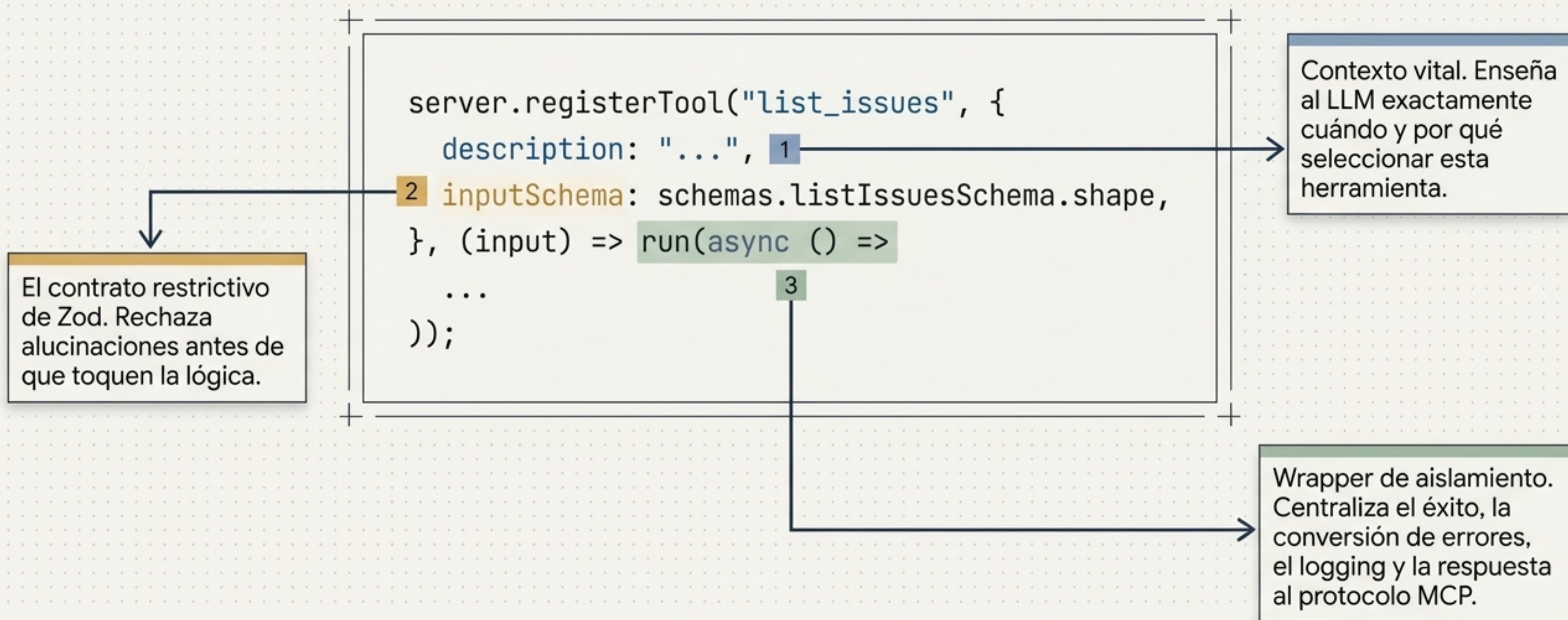
Trazabilidad completa: El recorrido de una petición MCP



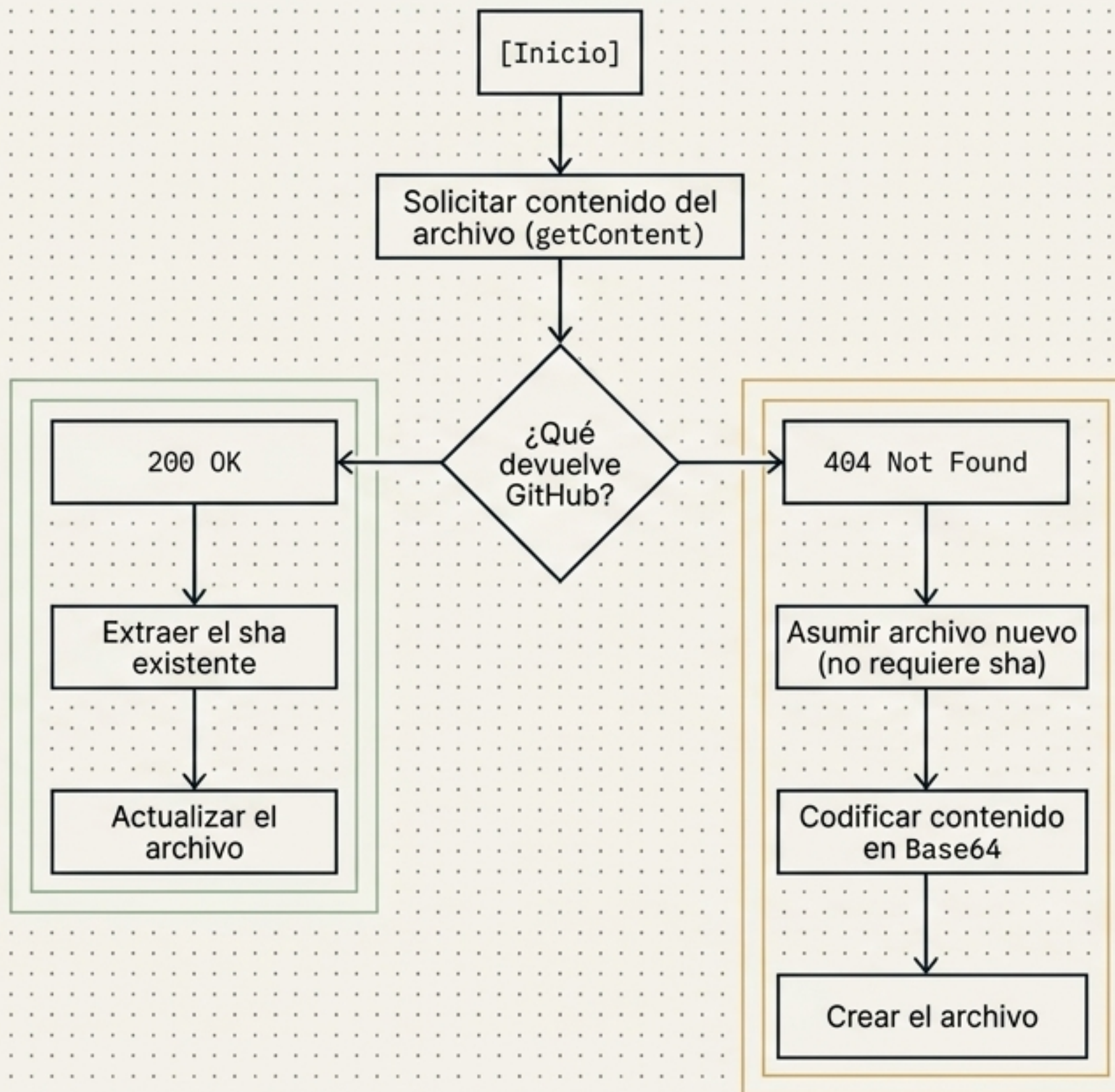
Racionalización del Stack Tecnológico

Tecnología	Responsabilidad	Por qué se eligió (Valor)
TypeScript	Contratos Explícitos	Reduce errores de desajuste entre schemas, herramientas y operaciones.
stdio	Transporte Local	Ejecutado directamente por el host; evita la sobrecarga y el riesgo de abrir puertos HTTP.
Zod	Frontera de Seguridad	Protege la API de alucinaciones del LLM validando nombres, rutas, estados y tamaños.
Octokit	Encapsulación	Maneja automáticamente autenticación, endpoints y retries sin construir peticiones manuales.
Arquitectura en Capas	Separación de Responsabilidades	server.ts aísla el protocolo MCP, schemas centraliza validación, operations.ts define la ejecución, errors normaliza respuestas.

Anatomía de un contrato: Conectando el LLM con la ejecución

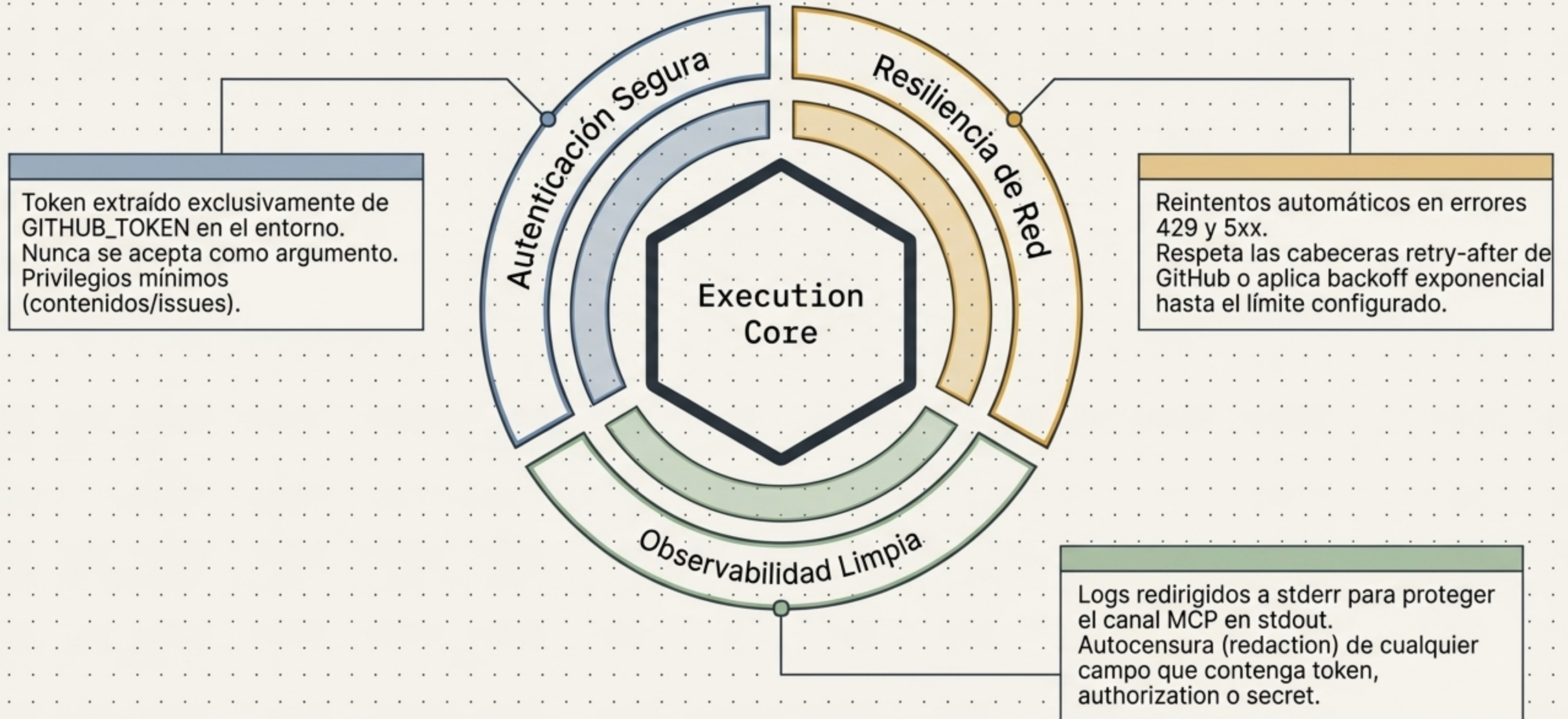


Desempaquetando la complejidad de create_commit



GitHub exige el SHA del blob actual para modificar un archivo, pero no para crearlo. El 404 no es un fallo, es una bifurcación funcional controlada (sujeta a normalización posterior para descartar errores de permisos).

Límites inquebrantables: Seguridad y Resiliencia



Estrategia de validación determinista sin efectos secundarios

El Alcance de las Pruebas

Total de 15 pruebas distribuidas en tres capas críticas:

4 pruebas de **Errores** (404,
• credenciales, red, fallos Zod).

5 pruebas de **Operaciones GitHub**
• (creaciones, actualizaciones con
SHA, flujos 404).

6 pruebas de **Schemas** (rechazo de
rutas inseguras, nombres cortos,
inputs inválidos).

El Entorno Simulado

Mocking: `Octokit` está
completamente interceptado.

Aisla la prueba del contrato (`Zod`)
y del comportamiento (normalización
de errores) sin tocar la API de
GitHub real.

Deuda Técnica Detectada

Faltan pruebas directas para
reintentos (`retry`), `logging` y el
registro nativo de herramientas.

Desafíos operativos superados

Problema		Solución Arquitectónica		Impacto
El canal MCP (`stdio`) se corrompe inmediatamente si el sistema emite logs de diagnóstico al proceso.	→	Redirigir todos los logs estrictamente a `stderr`.	→	Estabilidad del canal MCP y separación de flujo de datos y diagnóstico.
Los errores nativos de `Octokit` (ej. HTTP 403) son excesivamente crípticos para un agente conversacional.	→	Patrón `toAppError()`. Traduce fallos técnicos a mensajes accionables en lenguaje natural (ej. 'Verifica que el `owner/repo` exista').	→	Mensajes de error claros y accionables para el usuario, mejorando la experiencia.
Probar flujos complejos de creación y modificación dañaría repositorios reales de producción.	→	Inyección de dependencias para el cliente HTTP (`Mock rest`).	→	Entorno de pruebas seguro y determinista sin efectos secundarios en producción.

Flujo de ejecución para la demostración en vivo (Antigravity)

> **npm install && npm run build** (Pre-requisitos).

> Listar repositorios privados recientes (Validación de lectura).

> Crear repositorio temporal privado (Validación de escritura).

> Crear issue y hacer commit de archivo (Operaciones encadenadas).

> Comprobar validación de seguridad (Intentar acceder a **../secret.txt** y mostrar el rechazo).

> Ejecutar **npm test** (Demostrar suite sin efectos secundarios).

Regla de oro: NUNCA mostrar el valor del GITHUB_TOKEN en pantalla y destruir siempre el repositorio temporal al finalizar.

Decisiones de diseño frente al escrutinio

¿Por qué stdio y no HTTP?

stdio es perfecto para servidores lanzados localmente por el host MCP. Evita gestionar puertos o añadidos o añadir capas de autenticación de red innecesarias.

¿Qué ocurre ante un rate limit?

El servidor pausa temporalmente la ejecución respetando la cabecera retry-after. Si no existe, aplica backoff exponencial. Un 403 (permisos) aborta sin reintentos.

¿Cómo se evita filtrar el token?

El entorno inyecta la credencial; nunca cruza la barrera del agente. El sistema de logging redacta proactivamente cualquier campo sospechoso de ser un secreto.

¿Por qué mockear Octokit?

Garantiza pruebas ultrarrápidas y deterministas de la lógica de negocio y las validaciones de Zod, dejando la integración real exclusivamente para Antigravity.

Principios inmutables para herramientas controladas por LLM

El Modelo Servidor Agéntico Robusto

1. El Borde Estricto (Validación)

Un contrato de entrada implacable (Zod) es la única defensa efectiva contra las alucinaciones del agente antes de tocar una API externa.

2. Separación de Poderes (Ejecución)

El protocolo MCP delega la intención de negocio al agente, pero la autoridad sobre la ejecución, resiliencia y seguridad nunca abandona el servidor local.

3. Empatía en la Falla (Feedback)

Un error bien diseñado, normalizado y limpio no es un simple crash; es un prompt dinámico implícito que permite al LLM corregir su rumbo.